

New Project Setup Handbook

Bootstrapping any project on this machine — from an empty folder to a pushed GitHub repo open in VS Code — across every installed language and embedded toolchain.

Python

Node.js

Go

Rust

PlatformIO

ESP-IDF

Raspberry Pi Pico W

Contents

You only need **one** language section under Step 2 — jump straight to it via the bookmarks panel in your PDF viewer, or the color-coded index card on that page.

Before You Begin — GitHub CLI Setup	3
The Four-Step Workflow	4
Step 1 — Create the GitHub Repository	5
Step 2 — Set Up Your Project	6
Python	7
Node.js	8
Go	9
Rust	10
PlatformIO	11
ESP-IDF	12
Raspberry Pi Pico W	13
Step 3 — Connect to GitHub & Push	14
Step 4 — Open in VS Code	15
Daily Workflow Reminders	16
Appendix — gh CLI Cheat Sheet	17

ONE-TIME SETUP

Before You Begin — GitHub CLI Setup

Install and authenticate `gh` if not done yet:

```
brew install gh  
gh auth login
```

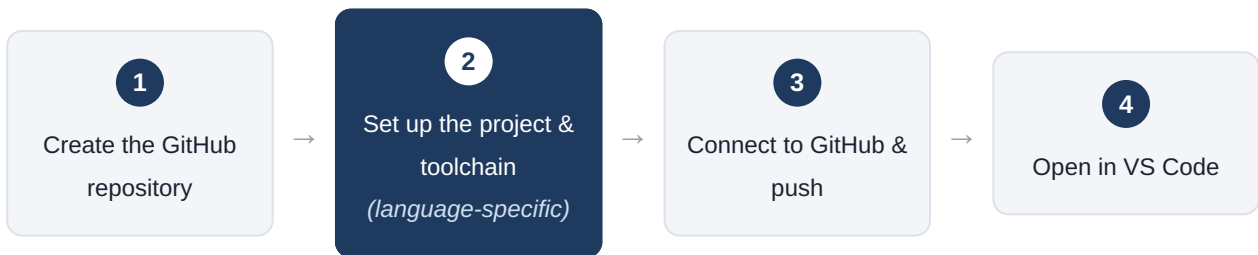
BASH

Choose **GitHub.com** → **HTTPS** → **Login with a web browser** and follow the prompts.

OVERVIEW

The Four-Step Workflow

Every project follows the same outer wrapper, regardless of language. Steps 1, 3 and 4 are identical no matter what you're building — only **Step 2** changes, and that's where the dedicated, color-coded chapters in this handbook come in.



STEP 1

Create the GitHub Repository

BASH

```
gh repo create my-project-name --private --description "Short description"
```

`--private / --public`

visibility

`--description`

optional but useful

**Don't pre-initialize**

Do

not

tick "Initialize repository" or add a README at this point — the local init in Step 2 does that.

STEP 2

Set Up Your Project

Pick the one section below that matches your stack — everything else in this handbook applies no matter which you choose.

UV-MANAGED**Python**

7

Fast, modern Python projects with uv for env + deps.

NPM / FNM**Node.js**

8

JavaScript or TypeScript, with automatic version switching.

GO MODULES**Go**

9

Single static binary, dependency isolation built in.

CARGO**Rust**

10

cargo scaffolds everything — including git — in one step.

EMBEDDED · MULTI-BOARD**PlatformIO**

11

Board-agnostic embedded framework for ESP32, AVR, and more.

ESPRESSIF**ESP-IDF**

12

The native Espressif framework for ESP32-family chips.

PICO SDK · CMAKE**Raspberry Pi Pico W**

13

C/C++ on RP2040/RP2350 using the official Pico SDK.

UV-MANAGED

Python

Fast, modern Python projects with uv for env + deps.

Create the project

BASH

```
cd ~/dev
mkdir my-project && cd my-project

uv init .          # creates pyproject.toml + hello.py stub
uv venv            # creates .venv/ (git-ignored by default)
source .venv/bin/activate
```

Add dependencies

BASH

```
uv add requests httpx      # runtime deps
uv add --dev pytest ruff    # dev-only deps
```

Run

BASH

```
python main.py
pytest
```



Already handled

`.venv/` is included in the `.gitignore` that `uv init` generates — nothing extra to configure.

NPM / FNM

Node.js

JavaScript or TypeScript, with automatic version switching.

Create the project

BASH

```
cd ~/dev
mkdir my-project && cd my-project

echo "24" > .node-version    # fnm auto-switches Node version on cd
node --version              # verify

npm init -y                 # creates package.json
```

Or, for a TypeScript project

BASH

```
npm init -y
npm install --save-dev typescript @types/node ts-node
npx tsc --init
```

Ignore dependencies

BASH

```
echo "node_modules/" >> .gitignore
```


GO MODULES

Go

Single static binary, dependency isolation built in.

Create the project

BASH

```
cd ~/dev
mkdir my-project && cd my-project

go mod init github.com/YOUR_USERNAME/my-project
```

Create `main.go`

GO

```
package main

import "fmt"

func main() {
    fmt.Println("hello")
}
```

Run

BASH

```
go run .
go build -o my-project .
```



Dependency isolation

Go modules handle this automatically via `go.mod` — no virtual environment needed.

CARGO

Rust

cargo scaffolds everything — including git — in one step.

Create the project

BASH

```
cd ~/dev
cargo new my-project
cd my-project
```

**Skip git init later**

`cargo new` creates the full project structure (`src/main.rs` , `Cargo.toml`) and already runs `git init` inside the directory. When you reach

Step 3

, skip the `git init` line — everything else still applies.

Run

BASH

```
cargo run
cargo test
cargo build --release
```

EMBEDDED · MULTI-BOARD

PlatformIO

Board-agnostic embedded framework for ESP32, AVR, and more.

Option A — VS Code (easiest)

1. Open VS Code, click the PlatformIO icon in the sidebar
2. **PlatformIO Home** → **New Project**
3. Set name, board (e.g. `Espressif ESP32-S3-DevKitC-1`), framework (`Arduino` or `ESP-IDF`)
4. Location: `~/dev/my-project`
5. Click **Finish** and wait for package download

Option B — CLI

```
cd ~/dev
mkdir my-project && cd my-project
pio project init --board esp32-s3-devkitc-1
```

Build and upload

```
pio run          # build
pio run --target upload
pio device monitor  # serial output
```

ESPRESSIF

ESP-IDF

The native Espressif framework for ESP32-family chips.



Activate every new terminal session

Run this once per terminal before any `idf.py` command:

```
get_idf
```

BASH

Create a project from the template

```
cd ~/dev
cp -r $IDF_PATH/examples/get-started/hello_world my-project
cd my-project
```

BASH

Or, from scratch in VS Code

1. Open VS Code → Command Palette (`Cmd+Shift+P`)
2. **ESP-IDF: New Project** → fill in name, target chip, location (`~/dev/`)

Build, flash, monitor

```
idf.py set-target esp32s3      # or esp32, esp32c3, etc.
idf.py build
idf.py -p /dev/cu.usbserial-* flash monitor
```

BASH



Multiple devices connected?

The port (`/dev/cu.usbserial-*`) auto-expands to the connected device. If several ports exist, use:

```
idf.py -p $(ls /dev/cu.usbserial-* | head -1) flash monitor
```

BASH

PICO SDK · CMAKE

Raspberry Pi Pico W

C/C++ on RP2040/RP2350 using the official Pico SDK.

Option A — VS Code Raspberry Pi Pico extension (recommended)

1. Open VS Code → Command Palette (`Cmd+Shift+P`)
2. **Raspberry Pi Pico: New Project from Example** — or **New C/C++ Project**
3. Set name (`my-project`), location (`~/dev/`), board (`Pico W`)
4. The extension generates `CMakeLists.txt`, `main.c`, and the toolchain config

Option B — manually

```
cd ~/dev
mkdir my-project && cd my-project
```

Create `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.13)
include($ENV{PICO_SDK_PATH}/external/pico_sdk_import.cmake)
project(my-project C CXX ASM)
pico_sdk_init()
add_executable(my-project main.c)
target_link_libraries(my-project pico_stdlib)
pico_add_extra_outputs(my-project)
```

Build

```
mkdir build && cd build
cmake .. -DPICO_BOARD=pico_w
make -j$(sysctl -n hw.logicalcpu)
```

Flash

Hold **BOOTSEL**, plug in USB, then copy the `.uf2` file:

```
cp my-project.uf2 /Volumes/RPI-RP2/
```

STEP 3

Connect to GitHub & Push

This step is identical for every project type, with one exception noted below.

```
# Inside ~/dev/my-project

git init
git add .
git commit -m "initial commit"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/my-project.git
git push -u origin main
```

**Exception: Rust**

`cargo new` already ran `git init` for you in Step 2 — skip that line. Everything else is identical:

```
git add .
git commit -m "initial commit"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/my-project.git
git push -u origin main
```

Shortcut via `gh`

Add the remote without copy-pasting the URL:

```
gh repo set-default YOUR_USERNAME/my-project
git push -u origin main
```

STEP 4

Open in VS Code



VS Code detects the language and activates the relevant extension automatically:

● Python	Picks up <code>.venv</code> and <code>pyproject.toml</code>
● Go	Detects <code>go.mod</code>
● Rust	Detects <code>Cargo.toml</code> and starts rust-analyzer
● PlatformIO	Shows the PlatformIO toolbar
● ESP-IDF	Shows the IDF status bar (chip target, port)
● Pico	Shows the Pico SDK status bar

QUICK REFERENCE

Daily Workflow Reminders

One block, three reminders — whichever apply to your stack:

```
cd ~/dev/my-project      # fnm switches Node version automatically

# Python – activate venv if not using uv run
source .venv/bin/activate

# ESP-IDF – activate in every new terminal
get_idf
```

BASH

REFERENCE

Appendix — gh CLI Cheat Sheet

TASK	COMMAND
Create GitHub repo	<code>gh repo create NAME --private</code>
Open repo in browser	<code>gh repo view --web</code>
View repo status	<code>gh repo view</code>
Create a PR	<code>gh pr create</code>
List open PRs	<code>gh pr list</code>
Clone an existing repo	<code>gh repo clone USER/REPO ~/dev/REPO</code>